

An Empirical Evaluation of Synchronization Mechanisms in Shared-Memory Parallel Systems

Pasupuleti T A N S V Vignesh
IIIT-Bangalore
IMT2023003

Kunda Sri Krishna Chaitanya
IIIT-Bangalore
IMT2023039

Abstract—Synchronization primitives are a fundamental building block of every concurrent application, yet their comparative performance characteristics across varied workloads, thread counts, and hardware configurations are rarely studied in a unified experimental framework. This paper presents a benchmark suite that evaluates four synchronization primitives, `std::mutex`, POSIX Semaphore, Test-and-Set SpinLock (TAS), and the MCS queue lock across six quantitative dimensions: throughput scalability, critical-section contention sensitivity, cache-line false-sharing overhead, acquisition fairness, tail-latency distribution, and throughput saturation under oversubscription. We apply the Gini coefficient to the distribution of per-thread lock acquisitions. This provides a single, scale-independent measure in the interval $[0, 1]$ that quantifies imbalance and potential starvation, independent of thread count and absolute acquisition rates. In addition to microbenchmarks, we validate findings on two real-world concurrent workloads, a striped-lock concurrent hash table and a bounded producer-consumer queue under realistic access patterns including read-dominant, write-dominant, bursty, and hotspot scenarios.

Index Terms—Synchronization Primitives, Mutual Exclusion, MCS Lock, SpinLock, Semaphore, Gini Coefficient, Lock Fairness, False Sharing, Concurrent Data Structures, Benchmarking, Multicore

I. INTRODUCTION

The correctness and performance of every multi-threaded application ultimately rests on its choice of synchronization primitive. From an operating-system scheduler to a web-server connection pool, the lock that guards a critical section determines throughput, latency, and fairness for all competing threads. Despite its importance, practitioners often choose `std::mutex` across workloads, largely due to the lack of clear, reproducible comparisons under diverse and realistic conditions.

Previous studies often evaluated isolated properties or targeted specialized hardware, limiting practical insight. A statistically rigorous comparison across scalability, fairness, contention, and workload characteristics on commodity systems remains limited.

This paper addresses that gap through a multi-experiment benchmark suite implemented in C++17. We deliberately select four primitives that span the design space along three axes, memory model (kernel vs. user-space), waiting strategy (blocking vs. spinning), and fairness guarantee (none, weak, FIFO) so that differences in experimental outcomes can be traced back to specific design decisions rather than

to incidental implementation details. The four primitives are: `std::mutex` (hybrid user/kernel, futex-based, weakly fair), POSIX Semaphore (pure kernel, weakly fair), Test-and-Set SpinLock (pure user-space, no fairness guarantee), and MCS Lock [2] (pure user-space, FIFO-fair, local spinning).

Our principal contributions are as follows.

- 1) A reproducible benchmark suite covering six experimental dimensions, scalability, contention sensitivity, false sharing, fairness, tail-latency distribution, and throughput saturation using statistically valid methodology: Welford online variance [8], Student *t*-distribution confidence intervals, and reservoir sampling for latency percentiles.
- 2) Application of the *Gini coefficient* [7] as a fairness metric for lock benchmarking, that provides a theoretically grounded, scale-independent starvation measure comparable across all thread counts and lock speeds.
- 3) Experimental validation on two real-world concurrent workloads (a concurrent hash table and a bounded producer-consumer queue) under six access-pattern scenarios each, bridging the gap between microbenchmarks and application-level performance.
- 4) Initial quantitative results from two completed experiments that confirm known theoretical properties empirically and assign concrete magnitudes to them.

The remainder of this paper is organised as follows. Section II surveys related work. Section III describes the benchmark methodology. Section IV presents initial experimental results.

II. RELATED WORK

A. MCS Lock and Queue-Based Locks

Queue-based locks, introduced by Mellor-Crummey and Scott [2], address the scalability limitations of simple test-and-set spinlocks by avoiding contention on a single shared memory location. The MCS lock achieves this by allowing each thread to spin on a private node, reducing coherence overhead and ensuring orderly access. As a result, it provides better scalability and guarantees starvation-freedom compared to conventional spinlocks.

Anderson [3] presented one of the earliest systematic empirical evaluations of spinlock algorithms, demonstrating that simple TAS locks suffer significant performance degradation

under contention due to cache-line interference. Subsequent queue-based designs, including CLH [4] [5] and MCS, mitigate this limitation by enabling local spinning and reducing coherence traffic. In this work, we focus on MCS due to its extensive treatment in the literature and its clearer correspondence to hardware-level behavior.

B. SpinLock (TAS) and Its Limitations

The Test-and-Set spinlock is the simplest mutual-exclusion primitive: a loop that atomically reads and sets a flag, exiting when the previous value was `false`. Anderson [3] showed that TAS performance degrades under contention due to repeated atomic operations on a shared cache line. Upon release, all waiting threads observe the change simultaneously—leading to the *thundering-herd* or *cache-thrashing* effect—as they retry concurrently and generate bursts of coherence traffic. The Test-and-Test-and-Set (TTAS) variant [9] mitigates this by using non-atomic reads before attempting the atomic operation, reducing unnecessary contention. In this work, we implement the simpler TAS variant to provide a clear baseline for isolating cache-coherence effects.

C. Gini Coefficient as a Fairness Metric

Fairness in lock benchmarking is most commonly reported as the max/min ratio of per-thread acquisition counts [6], or as the coefficient of variation of acquisition order. Both have weaknesses: the max/min ratio uses only two of n data points and is dominated by a single outlier thread, while the coefficient of variation is not bounded and cannot be compared across experiments with different total acquisition counts.

The *Gini coefficient* [7], a standard measure of distributional inequality, resolves both issues. For n threads with sorted per-thread acquisition counts $x_1 \leq x_2 \leq \dots \leq x_n$, it is defined as:

$$G = \frac{\sum_{i=1}^n (2i - n - 1) x_i}{n \sum_{i=1}^n x_i} \quad (1)$$

$G = 0$ indicates perfect equality—every thread acquires the lock equally often. $G = 1$ indicates complete starvation—one thread acquires all locks and the others none. The metric is scale-independent: it yields the same value regardless of whether locks complete 1,000 or 1,000,000 acquisitions per second, making it valid across all four primitives despite their very different throughputs. Its geometric interpretation via the Lorenz curve provides direct visual support for results.

The theoretical expectation for our four primitives follows directly from their designs. MCS Lock maintains an explicit FIFO queue, so $G \approx 0$. TAS SpinLock has no ordering mechanism, so the thread whose pipeline happens to be at the right stage on unlock repeatedly wins, producing a high G . `std::mutex` and Semaphore use kernel wait queues that approximate FIFO with OS-scheduler jitter, producing intermediate values.

D. Statistical Benchmarking Methodology

Following Georges et al. [10], we adopt a statistically rigorous methodology incorporating warmup phases, repeated trials, and confidence intervals to ensure reliable measurements. We use Welford’s algorithm [8] for numerically stable online computation of mean and variance, and report tail latencies (P95/P99) as recommended by Fleming and Wallace [12].

III. BENCHMARK METHODOLOGY

A. Primitives Selected

We evaluate four primitives chosen to maximally span the design space along three axes: memory model, waiting strategy, and fairness guarantee. Table I summarises their key properties.

TABLE I
SYNCHRONIZATION PRIMITIVES UNDER STUDY

Primitive	Space	Waits	Fairness	Traffic
<code>std::mutex</code>	User+Kernel	Block	Weak	$O(1)$
POSIX Semaphore	Kernel	Block	Weak	$O(1)$
SpinLock (TAS)	User	Spin	None	$O(P)$
MCS Lock	User	Spin	FIFO	$O(1)$

B. Experimental Dimensions

Six microbenchmark experiments are planned:

- 1) **Scalability** — Throughput (ops/sec) and mean latency (ns) vs. thread count $\in \{1, 2, 4, 8, 16, 32\}$ at critical-section (CS) work = 0 cycles.
- 2) **Contention / Critical Section Size** — Throughput vs. Critical Section work $\in \{0, 100, 1000, 10000\}$ busy-loop cycles at fixed 8 threads.
- 3) **False Sharing** — Throughput ratio of counters sharing one 64-byte cache line vs. `alignas(64)`-padded counters, at 2, 4, and 8 threads, with and without CPU affinity pinning.
- 4) **Fairness (Gini coefficient)** — Gini coefficient of per-thread acquisition counts at 2, 4, and 8 threads with a 100-cycle CS.
- 5) **Latency Distribution** — P50/P95/P99/P99.9 latency from reservoir-sampled distributions (up to 50,000 samples per thread) at 8 threads.
- 6) **Throughput Saturation** — Throughput sweep from 1 thread to $2\times$ hardware concurrency to locate the saturation point for each primitive.

C. Statistical Validity

Every configuration is repeated five times. A one-second warmup phase precedes each measurement window to allow CPU frequency scaling, TLB warm-up, and branch-predictor training to stabilise. Latency samples are collected using `CLOCK_MONOTONIC_RAW` timestamps bracketing each

acquire–release pair. Aggregate statistics use Welford’s algorithm [8] and 95% confidence intervals use the Student t -distribution with $n - 1$ degrees of freedom. Percentile latencies are computed from reservoir-sampled distributions to bound memory usage at $O(k)$ for a fixed reservoir size k .

IV. INITIAL EXPERIMENTAL RESULTS

Two of the six planned microbenchmark experiments have been completed: Experiment 1 (Scalability) and Experiment 2 (Contention / Critical Section Size). Experiments 3 to 6 and all real-world workload experiments are in progress.

A. Experiment 1: Lock Scalability

1) *Setup*: All four primitives were tested at thread counts $T \in \{1, 2, 4, 8, 16, 32\}$. The underlying system provides 12 logical cores (hardware threads), which serves as the baseline for scalability analysis. In particular, configurations with $T > 12$ represent oversubscription, where multiple software threads compete for the same core, potentially degrading performance due to context switching and scheduler contention. The critical section performs zero work to isolate pure lock overhead. Each configuration ran for 2 s with a 1 s warmup, repeated 5 times. Threads were assigned to distinct logical cores using `pthread_setaffinity_np` to eliminate OS-scheduling noise.

2) *Results*: Figure 1 presents throughput as a function of thread count. At $T = 1$, SpinLock achieves the highest throughput (3.80 Mops/sec) due to its minimal user-space overhead, slightly outperforming MCS Lock (3.60 Mops/sec). However, as thread count increases, the performance trends diverge significantly.

Within hardware concurrency ($T \leq 12$), MCS Lock achieves the highest throughput at all thread counts above $T = 1$, sustaining 2.25 Mops/sec at $T = 8$ compared to 1.25 Mops/sec for `std::mutex`, representing a $1.8\times$ improvement. This advantage arises from its $O(1)$ cache-coherence traffic, where each thread spins on a private cache line.

In contrast, SpinLock throughput declines sharply with increasing contention, falling to 1.75 Mops/sec at $T = 8$. This degradation is caused by repeated test-and-set retries that generate cache-line invalidation traffic proportional to the number of waiting threads.

Figure 2 shows mean acquisition latency as a function of thread count. SpinLock latency rises sharply with increasing contention, reaching approximately 60,000 ns at $T = 32$, indicating starvation due to the absence of fairness guarantees.

MCS Lock exhibits significantly higher latency at extreme contention levels, reaching approximately 1.15 ms at $T = 32$ with high variance ($\sigma \approx 200,000$ ns). This behavior is consistent with FIFO queue stalls, where preemption of predecessor threads delays all successors, resulting in latency amplification proportional to OS scheduling quanta.

Figure 3 shows normalized throughput relative to the single-thread baseline. At $T = 8$, MCS Lock retains approximately

60% of its baseline performance, outperforming SpinLock (47%), `std::mutex` (35%), and Semaphore (34%).

Beyond hardware concurrency ($T > 12$), MCS Lock degrades sharply, dropping to approximately 2% of its baseline at $T = 16$. This behavior confirms that queue-based locks are sensitive to oversubscription, where OS preemption can stall the FIFO queue and propagate delays to all waiting threads.

Key finding: *MCS Lock provides the best scalability under contention within hardware concurrency due to constant cache-coherence traffic, but degrades under oversubscription due to FIFO sensitivity to preemption. SpinLock offers the lowest overhead only at very low thread counts but becomes inefficient and prone to starvation as contention increases.*

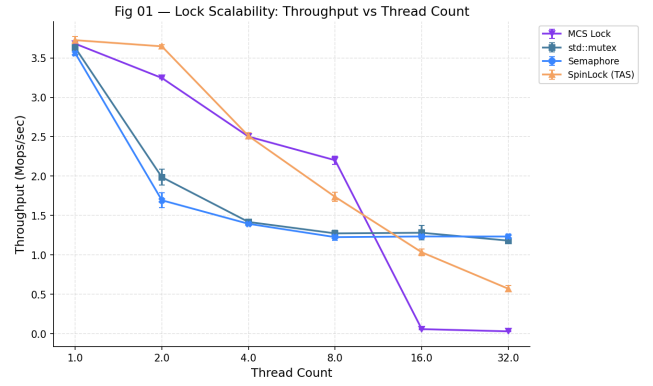


Fig. 1. Throughput vs thread count for all synchronization primitives.

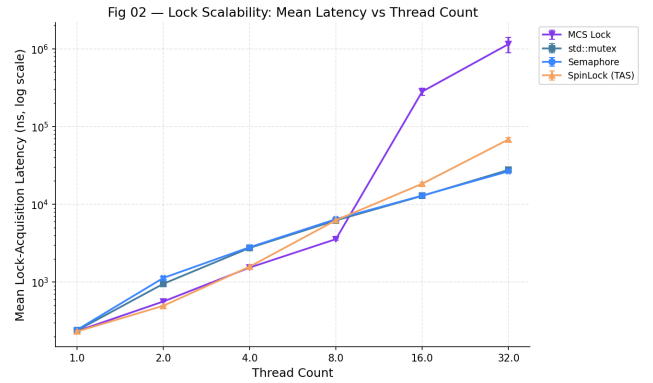


Fig. 2. Mean lock acquisition latency vs thread count.

B. Experiment 2: Contention / Critical-Section Size

1) *Setup*: This experiment evaluates the impact of critical-section (CS) size on lock performance under fixed contention. The number of threads is fixed at $T = 8$, corresponding to the number of available hardware cores, ensuring sustained contention without oversubscription effects.

The critical-section workload is varied using a synthetic busy-loop with cycle counts $\{0, 100, 1000, 10,000\}$. This

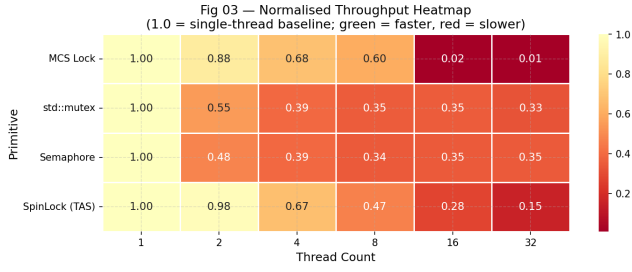


Fig. 3. Normalized throughput relative to single-thread baseline.

range spans from lock-dominated execution ($CS = 0$) to computation-dominated execution ($CS = 10,000$).

All other parameters remain consistent with Experiment 1: each configuration runs for 2 seconds with a 1 second warmup, repeated 5 times. Threads are pinned to distinct cores using `pthread_setaffinity_np` to eliminate scheduling noise.

2) *Results*: Figure 4 presents throughput as a function of critical-section (CS) size. At $CS = 0$, MCS Lock achieves the highest throughput (6.45 Mops/sec), a $1.85\times$ improvement over `std::mutex` (3.50 Mops/sec), due to its FIFO ordering eliminating wasted retry attempts. As Critical Section size increases, throughput for all primitives converges below 0.1 Mops/sec at $CS = 10,000$, where lock hold time dominates total execution cost.

Figure 5 shows P99 latency. A pronounced latency spike is observed for `std::mutex` and Semaphore at $CS = 500$ cycles (approximately 160,000 ns), exceeding both smaller and larger critical section sizes. This spike is attributed to a lock convoy effect, where multiple threads accumulate in the kernel wait queue and are awakened in bursts, resulting in excessive context-switch overhead. MCS Lock avoids this behavior due to strict FIFO wake-up, maintaining a significantly lower P99 latency of approximately 15,000 ns at $CS = 500$.

Figure 6 illustrates the crossover behavior between SpinLock and `std::mutex`. SpinLock outperforms `std::mutex` once critical section exceeds approximately 22 cycles, as the futex overhead is amortized. However, beyond approximately 7,491 cycles (approximately $2.1\ \mu s$), `std::mutex` regains the advantage as spinning overhead exceeds the cost of blocking and wake-up. This defines a practical operating window in which SpinLock is preferable.

Key finding: *SpinLock is advantageous only within a bounded critical-section window (22 to 7,491 cycles), while MCS Lock provides consistently stable performance across all critical section sizes. Blocking primitives such as `std::mutex` exhibit stable behavior at larger critical sections but are susceptible to lock convoy effects at intermediate contention levels.* Figures 4–6 summarise the impact

of critical-section size on throughput and latency across all primitives.

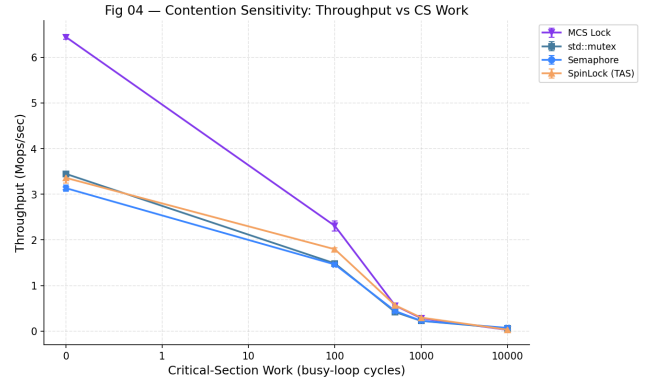


Fig. 4. Throughput vs critical-section size at $T = 8$ threads.

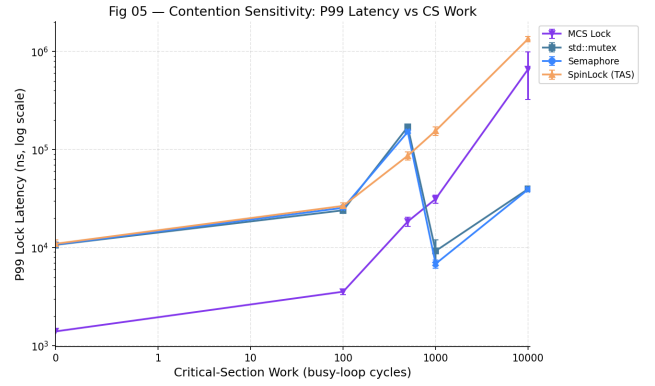


Fig. 5. P99 latency vs critical-section size highlighting lock convoy effects.

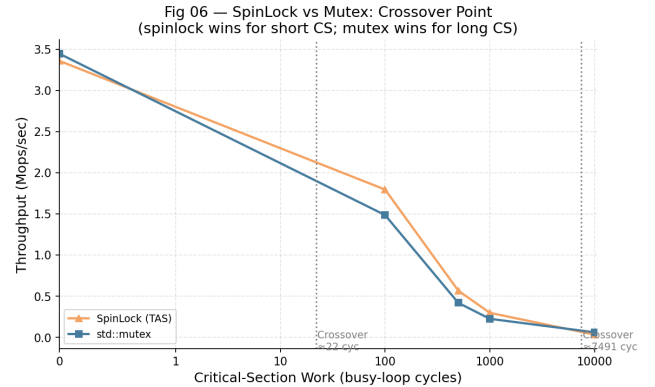


Fig. 6. Crossover region between SpinLock and `std::mutex` as a function of critical-section size.

C. Summary

Table II presents a qualitative comparison across the four primitives on the two completed experiments.

TABLE II
QUALITATIVE SUMMARY OF INITIAL EXPERIMENTAL RESULTS

Dimension	Mutex	Sem.	Spin	MCS
Scalability (high T)	Med.	Poor	Poor	Best
Throughput ($T = 1$)	Med.	Poor	Best	Good
Short Critical Section (0–100 cyc.)	Med.	Poor	Best	Good
Long Critical Section (>1000 cyc.)	Good	Good	Poor	Good

REFERENCES

- [1] O. Kode and T. Oyemade, “Analysis of synchronization mechanisms in operating systems,” arXiv:2409.11271 [cs.OS], Sep. 2024.
- [2] J. M. Mellor-Crummey and M. L. Scott, “Algorithms for scalable synchronization on shared-memory multiprocessors,” *ACM Trans. Comput. Syst. (TOCS)*, vol. 9, no. 1, pp. 21–65, Feb. 1991.
- [3] T. E. Anderson, “The performance of spin lock alternatives for shared-memory multiprocessors,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 1, no. 1, pp. 6–16, Jan. 1990.
- [4] T. S. Craig, “Building FIFO and priority-queueing spin locks from atomic swap,” Tech. Rep. TR 93-02-02, Univ. of Washington, Feb. 1993.
- [5] P. Magnusson, A. Landin, and E. Hagersten, “Queue locks on cache coherent multiprocessors,” in *Proc. 8th Int. Parallel Processing Symp. (IPPS)*, pp. 165–171, Apr. 1994.
- [6] M. L. Scott and W. N. Scherer, “Scalable queue-based spin locks with timeout,” in *Proc. 8th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, pp. 44–52, 2001.
- [7] C. Gini, “Measurement of inequality of incomes,” *The Economic Journal*, vol. 31, no. 121, pp. 124–126, Mar. 1921.
- [8] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, Aug. 1962.
- [9] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.
- [10] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 57–76, 2007.
- [11] U. Drepper, “Futexes are tricky,” Red Hat Inc., Tech. Rep., ver. 1.2, 2011. [Online]. Available: <http://www.akkadia.org/drepper/futex.pdf>
- [12] P. J. Fleming and J. J. Wallace, “How not to lie with statistics: The correct way to summarize benchmark results,” *Commun. ACM*, vol. 29, no. 3, pp. 218–221, Mar. 1986.
- [13] D. Vyukov, “Bounded MPMC queue,” 2010. [Online]. Available: <https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue>